

TOKENMAXXING VILLAGE / OFFLINE

From Prompt to Payload

Harnessing local models for offensive work.

All free software. Runs offline. No API keys, no telemetry,
nothing leaves the lab.

Milot Shala <https://milot.shala.tech/>

Two reasons, and only one of them is about preference.

- **The paperwork.** The context I would need to paste in belongs to a client, sits under an NDA, or describes infrastructure I am not allowed to discuss outside the engagement.
- **The network.** Segmented internal, no egress. Nothing gets out, and every packet I generate is something a defender might notice. A model that phones home is not an option.

The second one is not a cost argument. There is no price at which a hosted model becomes reachable from that box.

THE INVERSION

**Marginal cost per token is zero.
So capability is the only
currency left.**

- No budget to govern. No router tuned for price. No leaderboard to game.
- Which makes the honest question not “how many tokens did you burn” but **what does the cheap model actually get right.**
- That is a measurement, and it is the one everybody skips.

THE HARNESS

Small enough to audit before you point it at a client.

WHAT IS IN IT

```
payload/  
  agent.py      the loop  
  models.py     client + router  
  approval.py   deny by default  
  egress.py     offline guard  
  transcript.py append-only log  
  tools/        12 tools  
  rag/          sqlite retrieval
```

CHOICES I MADE

- **Zero required dependencies.** Standard library only. No package manager on the air-gapped box.
- **Roles, not model names.** plan / code / embed. What sits behind them is a provisioning decision.
- **Deny by default.** Nothing that touches a target runs unapproved.

Four things, and only one of them is a decision.

INSTALL

```
# Arch
sudo pacman -S python base-devel \
ollama radare2

# macOS
xcode-select --install
brew install python ollama radare2
brew services start ollama
```

WHAT EACH IS FOR

- **Python 3.11+** The harness itself. Nothing to pip install. Standard library only.
- **A C compiler** Builds the lab target and the egress interposer.
- **A model runtime** Ollama, llama.cpp, LM Studio or MLX. Any OpenAI-compatible endpoint on loopback.
- **radare2** Optional. Without it the tools fall back to objdump, nm and strings.

The model is yours to pick. Roles, not names. Weights move faster than workshops. 16 GB runs a 7-8B at 4-bit plus an embedder.

Everything has one entry point.

```
$ make
```

```
provision  phase 1: build lab, interposer, config (needs network once)
lab        build and verify the RE target only
test       agent loop + egress guard + lab bugs (no model, no network)
verify     everything in 'test' plus full egress verification
corpus     embed corpus/notes into the retrieval store
bundle     portable bundle for an air-gapped host
demo       pre-flight check to run before walking on stage
slides-pdf export the deck to PDF
clean      remove build artifacts and run output
```

make test needs no model and no network. It is how you check the install before anything is downloaded.

RUN IT YOURSELF / THREE COMMANDS

This is what I am about to type.

```
$ git clone <repo> && cd prompt-to-payload  
$ ./provision/provision.sh  
$ cd harness && python3 -m payload re \  
    ../lab/re-target/dist/vulnbox
```

- No model yet? Swap in the stub and the whole loop still runs:
`python3 harness/tests/stub_server.py 11455`
- Want to check it is really offline?
`./scripts/egress-test.sh`

DEMO

A stripped binary, and no source.

- `strings` gives it nothing. The interesting constant is XOR-encoded.
- `nm` gives it libc imports and `_main`. Every internal name is gone.
- It has to work from the disassembly, the imports, and the bytes in `__const`.

The source is in the repo. That is the whole point: we can score it.

SCORING IT / GPT-OSS:120B, 107 SECONDS

One right, one inverted, one never seen.

CORRECT

Stack buffer overflow. Found it, cited `0x100000858`, read the frame layout. Overstated the destination as 96 bytes; it is 64. Real analysis.

INVERTED

Off-by-one. Right function, backwards logic: claimed indices > 8 are accepted and ≤ 8 rejected. The opposite is true. Rated it **High confidence**, then invented a fourth finding from the mistake.

NEVER SEEN

Authentication bypass. Not mentioned once. Instead it described auth as matching the literal string "auth", which is the argv dispatch, not the key.

It did decode the XOR key at `0x52` from the disassembly. The capability is real. The confidence is not calibrated to it.

FOUR COMMANDS / GROUND TRUTH

Every confident claim, falsified from the shell.

it said > 8 is accepted and ≤ 8 rejected

```
$ ./dist/vulnbox slot 3 TEST    slot 3 set to TEST
```

```
$ ./dist/vulnbox slot 9 TEST    vulnbox: slot out of range
```

it said auth matches the literal string "auth"

```
$ ./dist/vulnbox auth auth      denied
```

it never mentioned this one at all

```
$ ./dist/vulnbox auth ""        authenticated
```

WHERE IT FALLS APART

The problem is not that it is wrong. It is that wrong looks like right.

- **Judgment on a live target.** It will propose the loud option as readily as the quiet one. It has no sense of what a defender sees.
- **Arithmetic on boundaries.** Off-by-one, bounds checks, index limits. It reasons about these confidently and gets them wrong.
- **Invented specifics.** Flags, offsets, function names. Fluent, plausible, absent from the binary.
- **Uniform confidence.** The tone of a verified claim and a guess are identical. That is what costs you time.

WHAT IT IS GENUINELY GOOD AT

Transformation, not judgment.

- **Recon triage.** Raw scanner output into a prioritised list.
You already ran the scan; this is the hour afterwards.
- **Reading unfamiliar code.** A language or codebase you did not write, at the speed of reading, not the speed of learning.
- **Translating between tool syntaxes.** The same intent expressed for a different tool, which is most of what muscle memory is for.
- **Turning output into prose a client can read.** The part of the engagement nobody enjoys and everybody bills for.

Provision once. Then nothing leaves, and you can check.

```
$ ./scripts/egress-test.sh

PASS interposer is active and intercepting (self-check)
PASS full agent loop completed (model call, tool dispatch, answer)
PASS 2 loopback operation(s) observed - the loop really ran
PASS zero non-loopback operations attempted
PASS control: a real egress attempt WAS detected and blocked
PASS with the lock on, no connection is even attempted

7 passed, 0 failed
```

Every *connect* and *sendto* intercepted at the libc boundary.
The control is there because a test that cannot fail is not evidence.

TAKE IT HOME

Clone it, run it with the wifi off.

- **Provision** needs network once. **Run** needs nothing, ever again.
- Ships a portable bundle for carrying onto an air-gapped host.
- The vulnerable source is included, so you can score your own model against the same three bugs.

Questions. All fifteen minutes of them.